RESEARCH-ARTICLE

EARN: Efficient Inference Acceleration for LLM-based Generative Recommendation by Register Tokens

CHAOQUN YANG, Tsinghua University, Beijing, China

XINYU LIN, National University of Singapore, Singapore City, Singapore

WENJIE WANG, University of Science and Technology of China, Hefei, Anhui, China

YONGQI LI, The Hong Kong Polytechnic University, Hong Kong, Hong Kong, Hong Kong

TENG SUN, Shandong University, Jinan, Shandong, China

XIANJING HAN, National University of Singapore, Singapore City, Singapore

View all

Published: 03 August 2025

Citation in BibTeX format

KDD '25: The 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining
August 3 - 7, 2025
Toronto ON, Canada

Conference Sponsors:
SIGMOD
SIGKDD

EARN: Efficient Inference Acceleration for LLM-based Generative Recommendation by Register Tokens

Chaoqun Yang
chaoqun@yang.emai.cn
Tsinghua University
Beijing, China

Xinyu Lin*
xylin1028@gmail.com
National University of Singapore
Singapore, Singapore

Wenjie Wang*
wenjiewang96@gmail.com
University of Science and Technology of China
Hefei, China

Yongqi Li
liyongqi0@gmail.com
The Hong Kong Polytechnic University
Hong Kong, China

Teng Sun
stbestforever@gmail.com
Shandong University
Qingdao, China

Xianjing Han
hanxianjing2018@gmail.com
National University of Singapore
Singapore, Singapore

Tat-Seng Chua
dcscts@nus.edu.sg
National University of Singapore
Singapore, Singapore

Abstract

Large Language Model-based generative recommendation (LLMRec) has achieved notable success, but it suffers from high inference latency due to massive computational overhead and memory pressure of KV Cache. Existing KV Cache reduction methods face critical limitations: cache compression offers marginal acceleration given recommendation tasks' short decoding steps, while prompt compression risks discarding vital interaction history. Through systematic analysis of attention patterns in LLMRec, we uncover two pivotal insights: 1) *layer-wise attention sparsity inversion* where early layers retain dense informative patterns while later layers exhibit high redundancy, and 2) *dual attention sinks phenomenon* where attention scores concentrate on both head and tail tokens of input sequences. Motivated by these insights, we propose **EARN**, an efficient inference framework that leverages the early layers to compress information into register tokens placed at the input sequence boundaries, then focuses solely on these tokens in the subsequent layers. Extensive experiments on three datasets, two LLMRec methods and two LLM architectures demonstrate EARN's superiority, achieving up to 3.79x speedup and 80.8% KV Cache reduction with better accuracy than the general finetuning approach. Our work bridges the efficiency-effectiveness gap in LLMRec, offering practical deployment advantages for industrial scenarios.

*Corresponding authors. This research is supported by the National Research Foundation, Singapore under its National Large Language Models Funding Initiative (AISG Award No: AISG-NMLP-2024-002). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.

KDD '25, Toronto, ON, Canada

ACM ISBN 979-8-4007-1454-2/2025/08
https://doi.org/10.1145/3711896.3736919

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

LLM-based Recommendation, Inference Acceleration, KV Cache

ACM Reference Format:
Chaoqun Yang, Xinyu Lin, Wenjie Wang, Yongqi Li, Teng Sun, Xianjing Han, and Tat-Seng Chua. 2025. EARN: Efficient Inference Acceleration for LLM-based Generative Recommendation by Register Tokens. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3736919

KDD Availability Link:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.15553291.

1 Introduction

Recently, Large Language Model (LLM)-based generative recommendation (LLMRec) has shown great potential due to its superior reasoning capabilities [10, 16, 20–22, 36, 38]. However, it suffers from high inference latency due to its massive model architecture and auto-regressive decoding paradigm, which severely limits its application in practical recommendation scenarios [12, 37, 44]. For instance, platforms like YouTube, which serve hundreds of millions of users daily with billions of interactions, require millisecond-level response times[1]. This highlights the stark efficiency gap between current LLMRec implementations and real-world application requirements. Therefore, enhancing the inference efficiency of LLMRec is a crucial issue for industrial applications.

For efficient inference of LLMRec, caching computed Key-Value state pairs (*i.e.,* KV Cache) is a popular choice. Technically, the inference pipeline with KV Cache typically consists of two distinct

[1]https://affmaven.com/youtube-statistics/

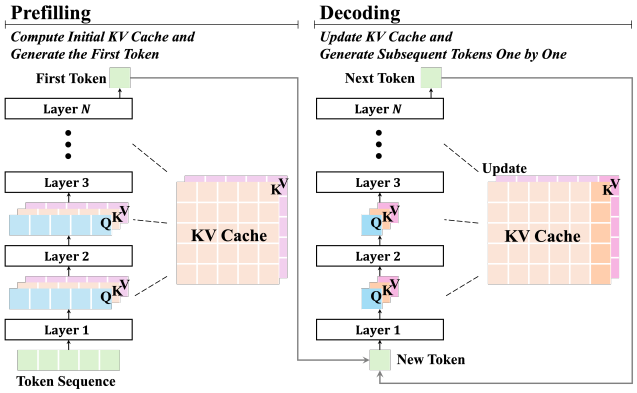

Figure 1: The inference process of LLMRec. In the *prefilling* stage, LLM computes the initial KV Cache and generates the first token of the output; in the *decoding* stage, LLM updates the KV Cache and generates subsequent tokens one by one.

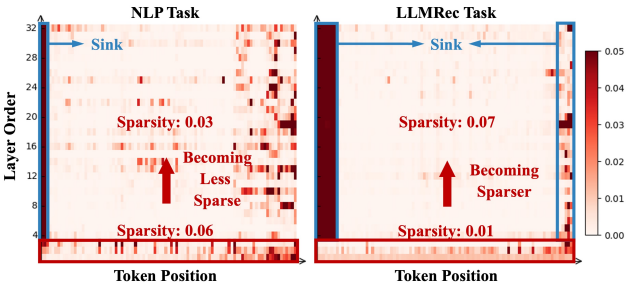

Figure 2: The attention distributions of different layers on the NLP task (reading comprehension QA) and the LLMRec task (LC-Rec on Beauty dataset) on Head 0 of Llama model. In the LLMRec task, the first three layers are relatively dense, while subsequent layers are sparse, with sinking occurring on the head and tail tokens.

stages. As shown in Figure 1, given the input token sequence (user profile, historical interactions, *etc.*), the first is the *prefilling* stage, where LLM generates the first token of the output and caches computed Key-Value state pairs of all layers for every token. The second is the *decoding* stage, where LLM generates subsequent tokens autoregressively using pre-computed KV Cache, and updates KV Cache in each decoding step. As the input sequence length increases, the computational overhead of KV Cache grows greatly, significantly prolonging the inference latency. And when the KV Cache size is large, each decoding step requires accessing a substantial amount of memory, which can severely slow down the inference. In light of the above, the key to accelerating the inference of LLMRec lies in reducing the size of KV Cache.

To address this challenge, some research has been proposed. One of the representative works is cache compression [27, 44]. It selectively removes less important KV pairs to reduce KV Cache size during *decoding*. However, recommendation tasks typically require few decoding steps to generate short output token sequences, such as item identifiers (around 1 to 5 tokens), limiting their acceleration potential in *decoding*. Another method is prompt compression [17, 44], which reduces initial KV Cache size during *prefilling* by condensating the input sequences, for instance, deleting unimportant tokens [14] or compressing the token sequence into a few tokens [18]. This technique not only decreases the computational load in *prefilling* but also alleviates the memory pressure in *decoding*. Although this method has achieved success in NLP [17, 44], it struggles to balance efficiency and accuracy in the context of LLMRec. In LLMRec, it is difficult to distinguish between important and unimportant items, which makes prompt compression method prone to discard crucial user interaction information, resulting in severe accuracy losses, or fail to achieve satisfactory acceleration in an effort to retain information. Hence, there is an urgent need for a method that enhances inference efficiency while preserving recommendation effectiveness.

To this end, we identify which information can be safely compressed in LLMRec, by inspecting the attention score distributions [40] in LLMRec. Along two critical dimensions: layer order and token position, we observe two key characteristics of LLMRec across three diverse datasets, two mainstream LLMRec methods, and two distinct LLM architectures. For example, as shown in Figure 2, on the Llama model and Beauty dataset, we found the following (more similar evidence can be found in Appendix A.1):

- **Layer-wise attention sparsity inversion.** Along the layer order dimension, we observe distinct sparsity between the NLP task and the LLMRec task. In the NLP task, the attention score distribution is sparse in the initial layers, but becomes relatively less sparse in the later layers (Sparsity: 0.06 → 0.03). Conversely, in the LLMRec task, the initial layers exhibit a relatively denser attention distribution, while the subsequent layers are highly sparse (Sparsity: 0.01 → 0.07).
- **Dual attention sinks phenomenon.** Along the token position dimension, while both the NLP task and the LLMRec task exhibit attention sinks [40], which means attention scores are highly concentrated on specific token positions, their distributions differ significantly. In the NLP task, sinks primarily emerge at the initial positions and a broad range of later tokens. In contrast, LLMRec demonstrates a distinctive dual concentration pattern - strong attention sinks at both head positions and a narrow tail region.

These findings suggest that in LLMRec: **1)** In terms of the layer order, the early layers contain richer information, while the middle tokens in the later layers are redundant. In fact, previous studies have shown that the early layers of LLMs are capable of summarizing required information and important tokens [32], while the later layers tend to be increasingly redundant, and pruning of redundant parts has minimal impact on performance [13]. **2)** In terms of the token position, the head and tail tokens carry the most important information. As explained in prior studies [7, 40], head tokens can absorb excess attention. Meanwhile, tail tokens interact with all previous tokens, which suggests that tail tokens of the prompt may possess the potential to summarize the preceding user interaction information.

Inspired by the above observations, we propose **EARN**, an **E**fficient inference **A**cceleration method for LLM-based **R**ecommendation by register toke**n**s (Figure 3), to enhance inference efficiency while maintaining recommendation effectiveness. Specifically, EARN introduces prefix and suffix register tokens - several learnable virtual tokens placed at the beginning and end of the input sequence, and

leverages the first k layers of LLM to compress the user's historical interaction information into register tokens. During inference, we use only register tokens for computation after k layers, thereby reducing the computational load and memory footprint of the KV Cache. We instantiate EARN on two mainstream LLMRec methods with two distinct LLM architectures, and conduct extensive experiments on three real-world datasets, validating the superiority of EARN in terms of both efficiency and accuracy. The code and datasets are available at https://github.com/transcend-0/EARN.

The contributions of our work are manifold:

- We identify the layer-wise attention sparsity inversion and dual attention sinks phenomenon in LLMRec, revealing that early layers as well as both head and tail tokens, retain the most critical information for LLMRec.
- We propose an efficient and effective method that leverages register tokens to compress user historical interactions and prune redundant computations in later layers, striking a delicate balance between inference efficiency and recommendation effectiveness.
- We validate the effectiveness of EARN through extensive experiments conducted on three real-world datasets, demonstrating the superiority of EARN in achieving both high inference speed and recommendation accuracy.

2 Preliminary

2.1 LLM-based Generative Recommendation

In this work, we primarily discuss the LLM-based generative recommendation. The main idea of LLM-based generative recommendation is using LLMs as the core recommender model, *i.e.,* taking the task instruction and a user's historical interactions as the input prompt to generate item identifiers, such as typical item IDs. Formally, given a user's historical interactions $H = (i_1, \cdots, i_T)$ in the chronological order, where i_1 to i_T represent the items the user has interacted with in the past, LLM-based generative recommendation predicts the next item i_{T+1} the user is likely to interact with, *i.e.,*

$$f(H) = P(i_{T+1} \mid i_1, \cdots, i_T) , \tag{1}$$

where f is the LLM, P is the probability distribution of the items.

2.2 Inference of LLMRec

LLM-based generative recommendation follows an auto-regressive decoding paradigm to generate item identifiers, which comprises two distinct computational stages: *prefilling* and *decoding*. Each stage exhibits unique inference characteristics and optimization requirements.

2.2.1 ***Prefilling.*** In this stage, the input prompt (containing the task instruction and the user's historical interactions) is tokenized into the token sequence and fed into the transformer decoder. The most pivotal component is the multi-head attention module that exists in each layer of the model. Within each attention head, the input hidden states $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ (n denotes the sequence length, and d denotes the hidden dimension) undergo linear transformations:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_{\boldsymbol{Q}}, \quad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_{\boldsymbol{K}}, \quad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_{\boldsymbol{V}} . \tag{2}$$

Subsequently, the attention mechanism computes the attention output as follows:

$$\boldsymbol{O} = Softmax\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d}}\right)\boldsymbol{V} . \tag{3}$$

where $\boldsymbol{O} \in \mathbb{R}^{n \times d}$ is the attention output, $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$ correspond to query states, key states, and value states, respectively. Since the generation of subsequent tokens utilizes key states $\boldsymbol{K}$ and value states $\boldsymbol{V}$ of preceding tokens, it is common practice to cache $\boldsymbol{K}$ and $\boldsymbol{V}$ for subsequent use, *i.e.,* KV Cache. This caching strategy is crucial for maintaining the efficiency of the inference process.

During the *prefilling* stage, $\boldsymbol{Q}$, $\boldsymbol{K}$ and $\boldsymbol{V}$ are all matrices, and matrix-matrix multiplication is compute-bound, meaning that the inference speed is primarily determined by the number of floating-point operations (FLOPs). Reducing the computational load can effectively accelerate the *prefilling* stage. For instance, shortening the input token sequence length can help decrease the quadratic computational complexity $O(n^2 d)$ of the attention mechanism. Besides, a shorter token sequence also reduces the size of KV Cache, thereby contributing to accelerating the *decoding* stage.

2.2.2 ***Decoding.*** In this stage, subsequent tokens are generated one by one, with the KV Cache being expanded accordingly. At each decoding step t, the model first computes $\boldsymbol{Q}_t, \boldsymbol{K}_t, \boldsymbol{V}_t \in \mathbb{R}^{1 \times d}$, then updates the KV Cache:

$$\boldsymbol{K}_{1:t} = Concat(\boldsymbol{K}_{1:t-1}, \boldsymbol{K}_t), \quad \boldsymbol{V}_{1:t} = Concat(\boldsymbol{V}_{1:t-1}, \boldsymbol{V}_t) . \tag{4}$$

Next, the attention output is calculated via $\boldsymbol{Q}_t \boldsymbol{K}_{1:t}^T$.

Unlike the *prefilling* stage, during the *decoding* stage, $\boldsymbol{Q}$ is a vector, while $\boldsymbol{K}$ and $\boldsymbol{V}$ are matrices. The vector-matrix multiplication is memory-bound, meaning that the inference speed is primarily limited by the efficiency of memory access rather than the computational capacity. The latency is thus predominantly determined by the efficiency of memory access to the growing KV Cache rather than FLOPs. In other words, reducing the size of KV Cache allows the GPU computing cores to access them more rapidly, which is the key to accelerating inference in the *decoding* stage.

3 Method

In order to improve the inference efficiency while ensuring the recommendation effectiveness, we proposed **EARN**, which achieves inference acceleration through register tokens. The overview of our method is presented in Figure 3.

3.1 Register Tokens

3.1.1 ***Prefix Register.*** We introduce the *prefix register*, *i.e.,* a set of learnable virtual tokens placed at the beginning of the input prompt. These tokens are designed to learn task-specific instructions, effectively signaling to the LLM that the current task is a recommendation problem. This method is inspired by the head sinks of the dual attention sinks phenomenon as elaborated in Section 1. Head sinks suggest that the head tokens can effectively divert attention away from task-irrelevant tokens in the subsequent layers. Drawing on the practice of prompt tuning [25, 26], where learnable virtual tokens are used to represent task instructions, we replace the BOS token, which the attention always sinks into,

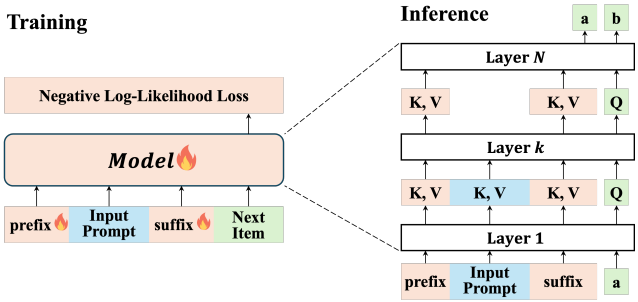

Figure 3: Overview of the proposed EARN. During training, the prefix register and the suffix register are also trainable. During inference, after layer k, EARN removes the prompt tokens to achieve acceleration.

with the learnable prefix register. This substitution not only fulfills the attention-diverting function of the BOS token, but also serves the role of task indication to tell the model that this is a recommendation task.

3.1.2 ***Suffix Register.*** We introduce the *suffix register*, *i.e.,* a set of learnable virtual tokens placed at the end of the input prompt, designed to summarize historical interactions and extract key information. This method is inspired by the tail sinks of the dual attention sinks phenomenon as elaborated in Section 1. The final tokens in a sequence have visibility over all preceding tokens, endowing them with summarization capabilities. Prior studies have observed that semantically meaningless tokens can effectively summarize preceding information during LLM inference [2, 30]. The occurrence of the tail sink phenomenon in the LLMRec task further corroborates this point. Building on this insight, we place learnable virtual tokens at the end of the input prompt and leverage the first k layers of the LLM to amplify this summarization capability.

3.2 Overall Pipeline

3.2.1 ***Training.*** During training, we employ next token prediction to train our model, computing the loss on the target item identifier, *i.e.,* minimizing the negative log-likelihood loss between the predicted next-item probabilities and the ground truth items. Formally, given a chronological sequence of user interactions $H = [i_1, i_2, ..., i_{T+1}]$, we construct the input sequence as:

$$X = [R_{\text{prefix}}; \text{Prompt}; R_{\text{suffix}}] \,, \tag{5}$$

$$Y = [i_{T+1}] \,, \tag{6}$$

where R_{prefix} and R_{suffix} denote the prefix register and the suffix register, respectively. Prompt is the user historical interactions $[i_1, i_2, ..., i_T]$ with the task instruction, and Y is the target item identifier i_{T+1}. The loss function is formulated as,

$$\mathcal{L} = -\sum_{j=1}^{|Y|} \log P(Y_j \mid X, Y_{<j}) \,, \tag{7}$$

where X is the input sequence of the sample, Y is the corresponding item identifier, Y_j is the j-th token in the item identifier Y, and $Y_{<j}$ denotes the tokens preceding Y_j in the item identifier.

When calculating the loss function, the main difference between our method and ordinary finetuning is the calculation of attention, where we exclude prompt tokens after layer k, *i.e.,*

- For layer $l \leq k$: All tokens participate in the attention calculation.
- For layer $l > k$: Only register and generated tokens participate in the attention calculation.

3.2.2 ***Inference.*** During inference, for a model with N layers, the generation of each token consists of two processes:

(1) **Full Computation Process (First k Layers)**: All tokens including task instructions, user historical interactions, prefix register and suffix register participate in the model computation. The model processes the complete input sequence to establish rich contextual representations.

(2) **Register-Focused Process (Remaining $N - k$ Layers)**: After k layers, we remove the original prompt tokens (task instructions and historical interactions), retaining only the register tokens and newly generated tokens. The attention mechanism in subsequent layers only operates on this reduced set of tokens:
   - In the *prefilling* stage, the input hidden state of layer k is transformed from $\boldsymbol{X} = [\boldsymbol{X}_{\text{prefix}}; \boldsymbol{X}_{\text{Prompt}}; \boldsymbol{X}_{\text{suffix}}]$ to $\boldsymbol{X'} = [\boldsymbol{X}_{\text{prefix}}; \boldsymbol{X}_{\text{suffix}}]$, significantly reducing the computational load and the initial KV Cache size.
   - In the *decoding* stage, the input hidden state of layer k is transformed from $\boldsymbol{X} = [\boldsymbol{X}_{\text{prefix}}; \boldsymbol{X}_{\text{Prompt}}; \boldsymbol{X}_{\text{suffix}}; \boldsymbol{X}_{\text{generated}}]$ to $\boldsymbol{X'} = [\boldsymbol{X}_{\text{prefix}}; \boldsymbol{X}_{\text{suffix}}; \boldsymbol{X}_{\text{generated}}]$, significantly reducing KV Cache that needs to be accessed during *decoding*.

This approach maintains the model's ability to leverage historical information through the learned register representations while avoiding redundant computations on lengthy prompt tokens.

3.3 Efficiency Analysis

Due to the removal of prompt tokens in the subsequent layers, we can significantly reduce both the computational load and memory footprint. Assume the model has N layers, n_h heads, an attention dimension of d_a, a hidden state dimension of d_h, and an intermediate dimension of d_f. The length of the input prompt is L, and the number of register tokens is $r \ll L$. If we remove the prompt tokens after k layers, the efficiency promotion can be analyzed through three key aspects:

- **Computation Complexity**: The FLOPs of the vanilla LLM are $N\ FLOPs_{LLM\ layer} = 4NL[n_h d_a(2d_h + L) + d_h d_f]$ (see Appendix A.2 for detailed derivation). In our EARN, it becomes $k\ FLOPs_{LLM\ layer} + (N-k)\ FLOPs_{Register\ layer}$. Thus, the attention complexity ratio γ_{attn} becomes:

$$\begin{aligned} \Gamma_{attn} &= \frac{k\ FLOPs_{LLM\ layer} + (N-k)\ FLOPs_{Register\ layer}}{N\ FLOPs_{LLM\ layer}} \\ &= \frac{k}{N} + (1 - \frac{k}{N}) \frac{r[n_h d_a(2d_h + r) + d_h d_f]}{L[n_h d_a(2d_h + L) + d_h d_f]} \\ &\approx \frac{k}{N}. \end{aligned} \tag{8}$$

- **KV Cache Size**: The memory size of the KV Cache is $M_{KV} N_{KV}$, where M_{KV} represents the memory size of each KV pair, and N_{KV} denotes the total number of KV pairs. The original N_{KV} would be $n_h NL$, whereas now it becomes $n_h(kL + (N-k)r)$.

Specifically, the KV Cache can be shrunk to $\frac{kL+(N-k)r}{NL}$ of its original size, reduced by $\frac{(N-k)(L-r)}{NL}$. That is to say, the KV cache size reduction ratio γ_{cache} is:

$$\begin{aligned}\Gamma_{cache} &= 1 - \frac{kL + (N-k)r}{NL} \\ &= \frac{(N-k)(L-r)}{NL} \\ &\approx \frac{N-k}{N}.\end{aligned} \tag{9}$$

- **Theoretical Speedup**: Assuming that FLOPS of the device is v_c, and HBM rate is v_m. The estimated time cost is

$$T = T_P + T_D = T_P + n_{generate} T_d \,, \tag{10}$$

where

$$T_P = \frac{FLOPs_{prefilling}}{v_c} \,, \tag{11}$$

$$T_d = max(\frac{Cache}{v_m}, \frac{FLOPs_{attn}}{v_c}) + \frac{FLOPs_{FFN}}{v_c} \,. \tag{12}$$

The theoretical speedup is

$$\Omega = \frac{T_{vanilla}}{T_{EARN}} \approx \frac{N}{k} \,. \tag{13}$$

For typical values ($N = 32,\ n_h = 32,\ d_a = 128,\ d_h = 4096, d_f = 11008,\ L = 512,\ k = 8,\ r = 2$), EARN reduces the KV Cache size to 75% of the original, and the speedup ratio can reach 4x.

4 Experiment

In this section, we conduct extensive experiments to answer the following research questions:

- **RQ1**: How does our proposed EARN perform compared to common inference acceleration methods?
- **RQ2**: How is the efficiency scalability of EARN under different batch sizes and sequence lengths?
- **RQ3**: How do different hyper-parameters affect the trade-off between inference efficiency and recommendation effectiveness of EARN?
- **RQ4**: How do different components contribute to EARN?

4.1 Experimental Settings

4.1.1 ***Models and Datasets***. We conduct experiments on two mainstream LLMRec methods: LC-Rec [43] and TIGER [31], with two distinct LLM architectures: Llama-7B [35] using multi-head attention and Qwen2.5-7B [34] using grouped-query attention. We test EARN on three real-world recommendation datasets: 1) **Beauty** contains user interactions with the beauty products. 2) **Games** covers user interactions with the video games. 3) **MovieLens-1M** collects user interactions with movies. More details of the datasets and experimental implementation details can be found in Appendix A.3.

4.1.2 ***Baselines***. We compare our approach against commonly used practices and existing SOTA methods, serving as our baselines:

- **Basic Methods**
  - *Finetune*: Standard full finetuning of the model.
  - *SkipLayers*: Skipping redundant subsequent layers.
- **Prompt Compression**: Methods targeting the *prefilling* stage.
  - *POD* [11]: Distilling task instructions in the prompt into few virtual tokens.
  - *500xCompressor* [18]: Compressing the context (user historical interactions in our scenario) within the prompt into one single special token, by employing frozen original LLM and trainable additional LoRA parameters.
- **Cache Compression**: Methods targeting the *decoding* stage.
  - *StreamingLLM* [40]: Statically retaining initial tokens and fixed-length recent tokens.
  - *SnapKV* [15]: Dynamically caching clustered important tokens.
- **Gist Methods**: Methods using the register token idea.
  - *Gist* [28]: Employing LLM itself to compress task instructions in the prompt into few gist tokens.
  - *AnLLM* [30]: Employing LLM itself to compress segmented sentences into the anchor token at the end of the sentence.

4.1.3 ***Evaluation Metrics***. Referring to previous work in the field of recommendation systems and LLM inference acceleration[8], we use the following metrics to evaluate our method:

- **Time Efficiency**
  - *Walltime Speedup* ω: The actual test speedup relative to vanilla auto-regressive decoding by comparing the wall clock time.
  - *Throughput* τ: The number of new tokens that the model generates per second.
- **Space Efficiency**
  - *KV Cache Reduction* γ: The empirically measured percentage reduction of the KV Cache.
  - *KV Cache Memory* σ: The average GPU memory usage (GB) of the KV Cache when generating the last token.
- **Recommendation Effectiveness**
  - *Recall@K* (*R@K*): A widely used measure of the model's ability to retrieve relevant items, which is defined as the proportion of relevant items that are recommended out of the total number of relevant items available.
  - *NDCG@K* (*N@K*): Normalized Discounted Cumulative Gain (NDCG) is a measure that takes into account both the order of relevant items and the relevance score of each item.

4.2 Overall Performance (RQ1)

The overall results of baselines and EARN instantiated on LC-Rec on three datasets and two LLM models are presented in Table 1. The results instantiated on TIGER are similar, which are moved to Appendix A.4. We draw a few observations as follows:

- Qwen-based implementations outperform Llama-based implementations in both inference efficiency and recommendation effectiveness. This superiority arises from: **1)** Qwen's grouped-head attention architecture achieves a smaller KV Cache and lower inference latency while maintaining semantic capability; **2)** Qwen's expanded tokenizer vocabulary (151,851 vs. Llama's 32,000) that shortens input prompts, reducing semantic fragmentation; **3)** Qwen is trained on a more massive dataset with approximately 18 trillion tokens, which provides it with a richer knowledge base and better language understanding capabilities.
- Among all inference acceleration baselines, although cache compression methods (StreamingLLM and SnapKV) achieve the highest cache reduction (over 90%), their end-to-end speedup is not as significant as prompt compression methods. This is because cache compression methods do not optimize the *prefilling* stage,

Table 1: Overall performance comparison between the baselines and EARN instantiated on LC-Rec. The best results are highlighted in bold and the second-best results are underlined.

Dataset	Model	Method	Time Efficiency		Space Efficiency		Recommendation Effectiveness			
			ω	τ	γ	σ	R@10	R@20	N@10	N@20
Beauty	Llama	**Finetune**	1.00	505.2	0.0	85.45	0.0145	0.0225	0.0084	0.0108
		SkipLayers	1.79	895.4	44.4	47.50	0.0013	0.0013	0.0013	0.0013
		POD	1.15	585.0	14.7	72.87	0.0045	0.0074	0.0032	0.0041
		500xCompressor	2.31	1168.6	74.8	21.55	0.0005	0.0006	0.0002	0.0003
		StreamingLLM	1.22	611.2	**96.4**	**3.09**	0.0005	0.0005	0.0004	0.0004
		SnapKV	1.20	600.7	94.5	4.73	0.0054	0.0061	0.0030	0.0032
		Gist	1.18	597.6	17.5	70.50	0.0048	0.0077	0.0028	0.0036
		AnLLM	1.24	625.1	92.2	6.70	0.0000	0.0000	0.0000	0.0000
		EARN	**3.79**	**1844.8**	80.5	16.68	**0.0167**	**0.0265**	**0.0095**	**0.0124**
	Qwen	**Finetune**	1.00	622.1	0.0	14.08	0.0145	0.0248	0.0087	0.0117
		SkipLayers	1.73	1056.1	58.0	5.92	0.0000	0.0000	0.0000	0.0000
		POD	1.09	679.3	8.4	12.90	0.0082	0.0127	0.0047	0.0061
		500xCompressor	2.56	1587.1	91.1	1.26	0.0003	0.0003	0.0001	0.0001
		StreamingLLM	1.05	652.6	**92.0**	**1.12**	0.0088	0.0147	0.0058	0.0075
		SnapKV	1.02	634.5	69.5	4.29	0.0097	0.0165	0.0058	0.0077
		Gist	1.15	715.4	20.0	11.30	0.0084	0.0161	0.0050	0.0074
		AnLLM	1.06	659.3	80.5	2.70	0.0000	0.0000	0.0000	0.0000
		EARN	**2.71**	**1662.6**	75.2	3.49	**0.0155**	**0.0265**	**0.0091**	**0.0122**
Games	Llama	**Finetune**	1.00	571.4	0.0	78.10	0.0167	0.0273	0.0106	0.0138
		SkipLayers	1.85	1045.3	45.0	42.93	0.0002	0.0002	0.0003	0.0003
		POD	1.09	623.3	15.9	65.69	0.0088	0.0147	0.0052	0.0070
		500xCompressor	2.11	1205.3	72.5	21.45	0.0014	0.0021	0.0009	0.0011
		StreamingLLM	1.23	702.5	**96.0**	**3.13**	0.0013	0.0013	0.0014	0.0014
		SnapKV	1.22	688.5	93.5	5.04	0.0102	0.0110	0.0070	0.0072
		Gist	1.12	639.9	18.0	64.00	0.0091	0.0146	0.0053	0.0070
		AnLLM	1.25	715.5	91.6	6.60	0.0003	0.0003	0.0003	0.0003
		EARN	**3.53**	**1930.6**	80.8	14.98	**0.0180**	**0.0291**	**0.0107**	**0.0142**
	Qwen	**Finetune**	1.00	568.1	0.0	13.03	0.0193	**0.0316**	**0.0127**	0.0155
		SkipLayers	1.52	858.4	57.3	5.57	0.0000	0.0000	0.0000	0.0000
		POD	1.35	767.5	8.1	11.98	0.0129	0.0209	0.0075	0.0100
		500xCompressor	2.60	1475.4	97.1	0.38	0.0013	0.0023	0.0008	0.0011
		StreamingLLM	1.05	594.7	**98.4**	**0.21**	0.0129	0.0202	0.0075	0.0098
		SnapKV	1.02	576.2	67.9	4.19	0.0138	0.0226	0.0083	0.0110
		Gist	1.41	801.3	22.9	10.00	0.0121	0.0154	0.0077	0.0088
		AnLLM	1.09	616.9	95.2	0.60	0.0003	0.0003	0.0003	0.0003
		EARN	**3.11**	**1711.3**	75.1	3.24	**0.0197**	0.0312	0.0122	**0.0157**
MovieLens	Llama	**Finetune**	1.00	704.3	0.0	55.39	0.0247	0.0449	0.0197	0.0288
		SkipLayers	2.52	1302.9	67.6	17.93	0.0022	0.0022	0.0043	0.0043
		POD	1.17	827.4	18.0	45.40	0.0066	0.0118	0.0062	0.0087
		500xCompressor	1.92	1352.8	61.1	21.54	0.0004	0.0005	0.0003	0.0003
		StreamingLLM	1.20	836.2	**95.2**	**2.66**	0.0000	0.0000	0.0000	0.0000
		SnapKV	1.19	829.2	91.9	4.50	0.0000	0.0000	0.0000	0.0000
		Gist	1.01	711.3	22.3	43.10	0.0232	0.0421	0.0118	0.0250
		AnLLM	1.20	846.9	89.5	5.80	0.0006	0.0008	0.0005	0.0006
		EARN	**3.21**	**2250.2**	79.7	11.26	**0.0259**	**0.0452**	**0.0247**	**0.0341**
	Qwen	**Finetune**	1.00	752.9	0.0	10.71	0.0289	0.0421	0.0247	0.0315
		SkipLayers	1.87	1124.7	76.6	2.50	0.0000	0.0000	0.0000	0.0000
		POD	1.41	1061.7	40.1	6.42	0.0139	0.0177	0.0115	0.0131
		500xCompressor	2.09	1577.8	77.6	2.40	0.0006	0.0009	0.0006	0.0007
		StreamingLLM	1.05	766.8	**88.9**	**1.19**	0.0258	0.0372	0.0197	0.0244
		SnapKV	1.00	731.9	76.5	2.52	0.0248	0.0428	0.0192	0.0266
		Gist	1.06	799.0	30.0	7.50	0.0287	0.0441	0.0244	0.0305
		AnLLM	1.03	772.4	75.9	2.60	0.0022	0.0022	0.0043	0.0043
		EARN	**2.84**	**2136.7**	66.7	3.56	**0.0298**	**0.0591**	**0.0252**	**0.0382**

whereas prompt compression methods (POD and 500xCompressor) reduce the input sequence length in the *prefilling* stage, thereby improving both computational and memory efficiency. This highlights the greater potential for acceleration in the *prefilling* stage for LLMRec. However, despite the notable inference efficiency gains of the baselines, they suffer from catastrophic recommendation effectiveness degradation. This highlights that user historical interactions in recommendation scenarios cannot be simply compressed at a high ratio using NLP methods.

- Although gist methods (Gist and AnLLM) also employ the idea of register tokens, they fail to achieve a significant speedup. This is because they utilize all layers of the LLM to compress information, resulting in substantial computational costs. In contrast, EARN only uses the first k layers, significantly reducing the

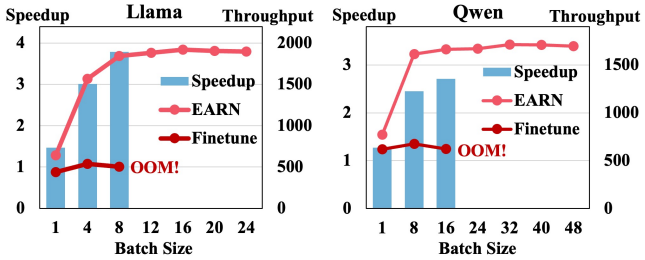

Figure 4: Efficiency under different batch sizes.

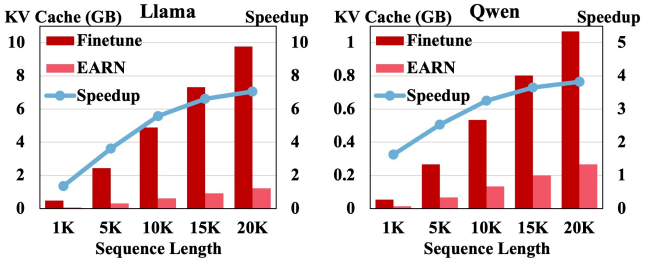

Figure 5: Efficiency under different sequence lengths.

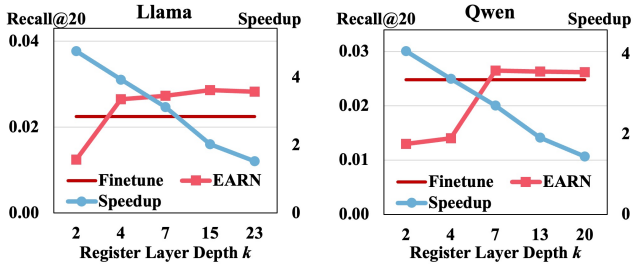

Figure 6: Effect of register layer depth k.

computational burden. In terms of space efficiency, Gist only compresses the task instruction within the input sequence, thus saving only about 20% of cache memory. Although AnLLM can reduce cache memory by 90%, it comes at the cost of catastrophic loss in recommendation effectiveness. AnLLM's recommendation effectiveness is nearly zero. This is due to its attempt to compress the intricate user interaction history into a single token, which is highly challenging and leads to severe information loss and poor performance. Gist shows a decline in recommendation effectiveness. This is because Gist compresses the task instruction in isolation within the language space, while the items in recommendations are not within the LLM's language space. Without direct interaction between the task instruction and historical items, the LLM struggles to understand the recommendation task's requirements to predict items outside its vocabulary. This impedes the method's applicability to recommendation tasks.

- EARN significantly achieves SOTA time efficiency (2.71-3.79x speedup) with excellent space efficiency (66.7-80.8% cache reduction), and also demonstrates a notable improvement over Finetune in terms of recommendation effectiveness. This suggests that the register tokens used in EARN can effectively summarize useful information, thereby ensuring that recommendation effectiveness is maintained while inference efficiency is improved.

4.3 Efficiency Scalability (RQ2)

To investigate the efficiency scalability of EARN, we evaluate the inference efficiency of EARN as compared to Finetune under varying batch sizes and sequence lengths.

4.3.1 ***Efficiency under Different Batch Sizes.*** To verify the acceleration effect of EARN under different batch sizes, we test the inference efficiency of EARN and Finetune with batch size in {1, 4, 8, $\cdots$, 24} on Llama and {1, 8, 16, $\cdots$, 48} on Qwen. As shown in Figure 4, EARN maintains significantly higher throughput than Finetune across all batch sizes. The speedup of EARN over Finetune increases as the batch size grows. Moreover, EARN can handle a larger batch size, which is particularly useful for practical scenarios. For example, on Llama, Finetune runs into out-of-memory (OOM) errors when the batch size exceeds 8, while EARN can still support larger batch sizes. The results clearly demonstrate that EARN significantly improves efficiency compared to Finetune across different batch sizes. Its ability to maintain high throughput on large batch sizes highlights its superior efficiency and practical inference capabilities. This makes EARN a more suitable choice for real-world applications where efficient and scalable inference is crucial.

4.3.2 ***Efficiency under Different Sequence Lengths.*** In industrial settings, user interaction histories can be extensive. Therefore, it is essential to assess the efficiency of EARN on longer sequences. To simulate this scenario, we pad our dataset to specific lengths (1K, 5K, $\cdots$, 20K) to test the acceleration effects. From Figure 5, we can see EARN consistently uses significantly less KV Cache than Finetune across all sequence lengths. Moreover, the larger the sequence length, the more significant the acceleration effect, achieving a speedup of up to 7x when the sequence length is 20K on Llama. This is highly desirable in practice, especially for long-term sequential recommendation, where user historical interaction sequences are stored for a long time, thus learning user preferences comprehensively. The results clearly demonstrate that EARN significantly improves efficiency compared to Finetune across different sequence lengths. Its ability to maintain a low KV Cache size and achieve higher speedup, especially for longer sequences, highlights its superior efficiency and practical inference capabilities. This makes EARN a more suitable choice for real-world applications where efficient and scalable inference is crucial, particularly when dealing with long sequences.

4.4 Hyper-parameter Analysis (RQ3)

To investigate the trade-off between inference efficiency and recommendation effectiveness caused by variations in hyper-parameters, we train and evaluate EARN with different register layer depth k and register token number n.

4.4.1 ***Effect of Register Layer Depth k.*** Figure 6 reveals the trade-off between inference efficiency and recommendation effectiveness when varying the register layer depth k: **1)** Too shallow: When the register layer depth is too shallow (*e.g.*, < 4 on Llama, and < 7 on Qwen), although the speedup is high, the recall suffers a significant loss. **2)** Optimal range: In the range of $k = 4 - 7$, EARN strikes a balance between speedup and recall. The speedup remains reasonably high, while Recall@20 improves notably. This suggests

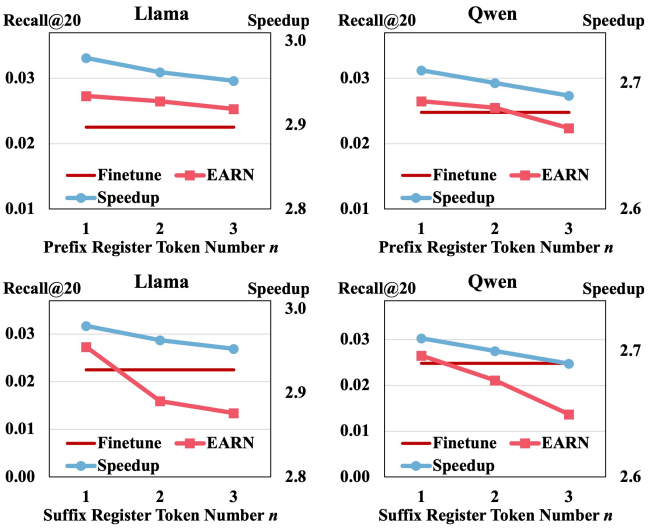

Figure 7: Effect of register token number n.

Figure 8: Effect of register token number n under different prompt length.

that the model can effectively capture the necessary information for recommendations without a substantial loss in inference efficiency. **3)** Too deep: When the register layer depth is too deep (*e.g.*, $k \geq 13$), the speedup decreases significantly, but Recall@20 does not show a proportional improvement. For example, on Qwen, the speedup drops sharply when $k = 13$ or $k = 20$, while Recall@20 does not increase substantially. This indicates that increasing the register layer depth beyond a certain point leads to diminishing returns in terms of recommendation effectiveness while significantly compromising inference efficiency.

4.4.2 ***Effect of Register Token Number n.*** Figure 7 reveals the trade-off between inference efficiency and recommendation effectiveness when varying the register token number n, from which we can observe that: **1)** For both the prefix register and the suffix register, as the register token number n varies from 1 to 3, the speedup gradually decreases. This is primarily because, as the register token number increases, the size of the KV Cache that needs to be computed and cached also grows, leading to increased inference latency. **2)** There is a decline in Recall@20 as the register token number n increases for both the prefix register and the suffix register. This phenomenon may be attributed to the interactions among multiple tokens in the later layers of the model, which can lead to a progressive distortion of the summarized information. The impact of this accuracy degradation is relatively minor for the prefix register but more pronounced for the suffix register. This discrepancy is likely due to the fact that user interaction history summarized by the suffix register is more complex and challenging to condense compared to task instructions summarized by the prefix register.

Although the above results show that a single register token is optimal, as prompt length increases, the optimal number of register tokens may need to be adjusted to maintain the performance. Therefore, we conduct further experiments to investigate how EARN performs under different prompt lengths and whether the optimal number of register tokens should be adjusted as the prompt length increases. We segmented the dataset into three groups based on prompt length (50-100, 100-150, and 150-200 tokens) to evaluate our EARN's performance across different prompt lengths. As shown in Figure 8, EARN consistently outperforms the Finetune baseline, and a single register token remains optimal across all prompt lengths. However, we observed an interesting trend for the suffix register token: the performance gap between 1 and 2 register tokens narrows as prompt length increases. This suggests that the optimal number of register tokens may rise for very long prompts. Nevertheless, experimental results on three real-world datasets (Table 1) demonstrate that employing a single suffix register token could achieve excellent recommendation performance, which is applicable to the majority of recommendation scenarios.

4.4.3 ***Hyper-parameter Recommendation.*** We experimentally validated that EARN's hyper-parameters can be selected through simple heuristics rather than exhaustive tuning. And we validated that setting the register layer and using a single register token at one-fourth achieves significant acceleration without accuracy loss on three real-world datasets. This configuration is broadly applicable for most recommendation tasks. Detailed analysis is as follows: **1)** Register layer depth k: Firstly, the lower the register layer k, the greater the acceleration effect achieved. Secondly, our sensitivity analysis indicates that there is essentially no loss in recommendation effectiveness when k is set to at least one-fourth of the total layers. **2)** Register token number n: Across two distinct LLM models (Llama and Qwen), we consistently found that one register token is optimal. Additionally, to assess if the number needs adjustment as prompt length increases, we conducted grouped experiments by length, and found that a single register token remains optimal under different prompt lengths. Thus, for most LLMRec scenarios, we propose a default configuration:

(1) ***k*:** One-fourth of the total layers as the register layer depth.
(2) ***n*:** One prefix register token and one suffix register token.

This setting could achieve a favorable speedup and KV Cache reduction while maintaining strong recommendation performance. And it's easily adjustable for various deployment scenarios.

4.5 Ablation Studies (RQ4)

To study the contribution of each component of the proposed EARN, we conduct ablation studies on the register training strategy and the register token.

4.5.1 ***Effect of Register Training.*** We first investigate the necessity of register training by comparing it with direct register inference on fully finetuned models. As shown in Table 2, models without RT suffer significant performance degradation across all metrics. Specifically for Llama with $k = 7$, removing RT leads to 72%

Table 2: Ablation study of the *register training* (RT).

Model	k	Method	R@10	R@20	N@10	N@20
Llama	7	Finetune	0.0145	0.0225	0.0084	0.0108
		EARN	0.0174	0.0273	0.0098	0.0127
		w/o RT	0.0048	0.0060	0.0021	0.0025
	15	EARN	0.0168	0.0286	0.0093	0.0127
		w/o RT	0.0053	0.0083	0.0029	0.0038
Qwen	7	Finetune	0.0145	0.0248	0.0087	0.0117
		EARN	0.0155	0.0265	0.0091	0.0122
		w/o RT	0.0074	0.0093	0.0044	0.0050
	13	EARN	0.0156	0.0263	0.0098	0.0129
		w/o RT	0.0072	0.0095	0.0043	0.0050

Table 3: Ablation study of the *prefix register* (PR) and the *suffix register* (SR).

Model	k	Method	R@10	R@20	N@10	N@20
Llama	7	EARN	0.0174	0.0273	0.0098	0.0127
		w/o PR	0.0172	0.0252	0.0108	0.0132
		w/o SR	0.0041	0.0075	0.0021	0.0031
	15	EARN	0.0168	0.0286	0.0093	0.0127
		w/o PR	0.0166	0.0260	0.0103	0.0131
		w/o SR	0.0074	0.0119	0.0035	0.0047
Qwen	7	EARN	0.0155	0.0265	0.0091	0.0122
		w/o PR	0.0153	0.0217	0.0097	0.0116
		w/o SR	0.0044	0.0071	0.0023	0.0031
	13	EARN	0.0156	0.0263	0.0098	0.0129
		w/o PR	0.0153	0.0230	0.0102	0.0124
		w/o SR	0.0065	0.0106	0.0036	0.0049

relative drop in R@10 (from 0.0174 to 0.0048) and 80% reduction in N@20 (from 0.0127 to 0.0025). This demonstrates that directly applying register inference without dedicated training fails to effectively capture task-specific knowledge. With RT, both models achieve consistent improvements over Finetune. Llama with $k = 7$ gains 17% higher R@10 (0.0174 vs. 0.0145) and 15% better N@20 (0.0127 vs. 0.0108). This indicates that register training enables the model to summarize key information from user historical interactions into the register token.

4.5.2 ***Effect of Prefix Register and Suffix Register.*** We further dissect the contribution of the prefix register and the suffix register through component-wise ablation. Table 3 reveals three key observations: **1)** Suffix register dominates performance: Removing the suffix register (w/o SR) causes significant performance collapse for both Llama and Qwen. This confirms our design intuition that summarizing historical interactions in the suffix register is crucial for recommendation tasks. **2)** Prefix register enhances task awareness: Though not as severe as removing the suffix register (w/o SR), removing the prefix register (w/o PR) still leads to a certain degree of performance degradation. This suggests the prefix register effectively primes the model for recommendation task identification. **3)** Layer-depth interaction: The impact of removing the suffix register is more pronounced in the lower layers (*e.g.,* $k = 7$ exhibits a greater performance drop than $k = 15$ on Llama), indicating that the suffix register plays a more significant summarization role in the lower layers. The lower layers rely more heavily on the compressed historical interaction information encoded in the suffix register.

5 Related Work

- **Inference Acceleration of LLMRec**. While LLM-based generative recommendations have shown remarkable performance [10, 16, 20–22, 36, 38], their practical application is hindered by high inference latency [12, 37, 44]. To tackle this issue, various research has been proposed. Several techniques leverage knowledge distillation to transfer comprehensible knowledge [4] or abstract knowledge [33] from a teacher LLM to a smaller student language model. Additionally, some methods apply speculative decoding to achieve lossless decoding acceleration [23, 39]. Besides, some approaches attempt to design efficient attention mechanisms to reduce computational complexity, such as sparse attention [6], linear attention [24], and slimming architecture [19].
- **KV Cache Reduction**. It's common practice to utilize KV Cache to reduce redundant computations during LLM inference. However, the use of KV Cache introduces new challenges. KV Cache will increase linearly with the length of the sequence, and the memory required will become larger and larger. To address this challenge, diverse methods have been proposed to reduce KV Cache, including two primary groups of work, *i.e.,* prompt compression and cache compression. Prompt compression reduces the initial KV Cache size during *prefilling* by shortening the input token sequence. Hard compression methods like SelectiveContext [14] and LLMLingua [9] filter redundant tokens while preserving natural language syntax, albeit at the cost of fluency. Soft compression techniques, such as AutoCompressor [3] and 500xCompressor [18], encode prompts into dense latent tokens, achieving higher compression ratios but sacrificing human interpretability. Cache compression reduces KV cache during *decoding*. Existing work employs eviction or merging strategies. Evict-based methods like StreamingLLM [40] and SnapKV [15] selectively evict less important KV Cache through specific rules. Merge-based approaches (*e.g.,* CAM [42] and DMC [29]) adaptively merge to-be-evicted caches into the remaining ones.

6 Conclusion

In this work, we address the critical challenge of inference efficiency in LLM-based recommendation systems (LLMRec), where the massive computational overhead and memory pressure of KV Cache severely hinders practical deployment. Through systematic analysis of LLMRec's attention patterns, we identify two pivotal characteristics: 1) layer-wise attention sparsity inversion, where in early layers retain dense informative patterns while later layers exhibit high redundancy, and 2) the dual attention sinks phenomenon, where attention scores concentrate on both head and tail tokens of input sequences. These insights motivate our proposed EARN method, which introduces prefix and suffix register tokens to compress task instructions and user interaction histories, implementing layer-wise computation pruning. EARN achieves an 80% reduction of KV Cache while maintaining essential information integrity. Extensive experiments conducted on three benchmark datasets and two distinct LLM architectures reveal that our EARN attains 3.79x inference acceleration with superior accuracy compared to conventional finetuning approaches. This breakthrough effectively reconciles the longstanding trade-off between inference efficiency and recommendation quality in LLMRec, presenting tangible deployment benefits for industrial-scale recommendation services.

References

[1] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. 2021. Scatterbrain: Unifying sparse and low-rank attention. *Advances in Neural Information Processing Systems* 34 (2021), 17413–17426.

[2] Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. 2024. SepLLM: Accelerate large language models by compressing one segment into one separator. *arXiv preprint arXiv:2412.12094* (2024).

[3] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3829–3846.

[4] Yu Cui, Feng Liu, Pengbo Wang, Bohao Wang, Heng Tang, Yi Wan, Jun Wang, and Jiawei Chen. 2024. Distillation matters: empowering sequential recommenders to match the performance of large language models. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 507–517.

[5] Yichuan Deng, Zhao Song, Jing Xiong, and Chiwun Yang. 2024. How Sparse Attention Approximates Exact Attention? Your Attention is Naturally n^C-Sparse. *arXiv preprint arXiv:2404.02690* (2024).

[6] Xinyan Fan, Zheng Liu, Jianxun Lian, Wayne Xin Zhao, Xing Xie, and Ji-Rong Wen. 2021. Lighter and better: low-rank decomposed self-attention networks for next-item recommendation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1733–1737.

[7] Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. 2025. When attention sink emerges in language models: An empirical view. In *The 13th International Conference on Learning Representations*.

[8] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. In *The 9th International Conference on Learning Representations*.

[9] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 13358–13376.

[10] Sein Kim, Hongseok Kang, Seungyoon Choi, Donghyun Kim, Minchul Yang, and Chanyoung Park. 2024. Large language models meet collaborative filtering: An efficient all-round LLM-based recommender system. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1395–1406.

[11] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Prompt distillation for efficient LLM-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1348–1357.

[12] Lei Li, Yongfeng Zhang, Dugang Liu, and Li Chen. 2024. Large language models for generative recommendation: A survey and visionary discussions. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. 10146–10159.

[13] Pengxiang Li, Lu Yin, and Shiwei Liu. 2025. Mix-LN: Unleashing the power of deeper layers by combining Pre-LN and Post-LN, In The 13th International Conference on Learning Representations. *arXiv preprint arXiv:2412.13795*.

[14] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 6342–6353.

[15] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. SnapKV: LLM knows what you are looking for before generation. *Advances in Neural Information Processing Systems* 37 (2024), 22947–22970.

[16] Yongqi Li, Xinyu Lin, Wenjie Wang, Fuli Feng, Liang Pang, Wenjie Li, Liqiang Nie, Xiangnan He, and Tat-Seng Chua. 2024. A survey of generative search and recommendation in the era of large language models. *arXiv preprint arXiv:2404.16924* (2024).

[17] Zongqian Li, Yinhong Liu, Yixuan Su, and Nigel Collier. 2024. Prompt compression for large language models: A survey. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 7182–7195.

[18] Zongqian Li, Yixuan Su, and Nigel Collier. 2024. 500xCompressor: Generalized prompt compression for large language models. *arXiv preprint arXiv:2408.03094* (2024).

[19] Jianghao Lin, Xinyi Dai, Rong Shan, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Large language models make sample-efficient recommender systems. *Frontiers of Computer Science* 19, 4 (2025), 194328.

[20] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2025. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems* 43, 2 (2025), 1–47.

[21] Xinyu Lin, Wenjie Wang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Bridging items and language: A transition paradigm for large language model-based recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1816–1826.

[22] Xinyu Lin, Wenjie Wang, Yongqi Li, Shuo Yang, Fuli Feng, Yinwei Wei, and Tat-Seng Chua. 2024. Data-efficient fine-tuning for LLM-based recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 365–374.

[23] Xinyu Lin, Chaoqun Yang, Wenjie Wang, Yongqi Li, Cunxiao Du, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2025. Efficient inference for large language model-based generative recommendation. In *The 13th International Conference on Learning Representations*.

[24] Langming Liu, Liu Cai, Chi Zhang, Xiangyu Zhao, Jingtong Gao, Wanyu Wang, Yifu Lv, Wenqi Fan, Yiqi Wang, Ming He, et al. 2023. Linrec: Linear attention mechanism for long-term sequential recommender systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 289–299.

[25] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. 2022. P-tuning v2: Prompt tuning can be comparable to fine-tuning universally across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. 61–68.

[26] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. 2024. GPT understands, too. *AI Open* 5 (2024), 208–215.

[27] Shi Luohe, Hongyi Zhang, Yao Yao, Zuchao Li, et al. 2024. Keep the cost down: A review on methods to optimize LLM's KV-Cache consumption. In *The 1st Conference on Language Modeling (COLM)*.

[28] Jesse Mu, Xiang Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. *Advances in Neural Information Processing Systems* 36 (2023), 19327–19352.

[29] Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo M Ponti. 2024. Dynamic memory compression: retrofitting LLMs for accelerated inference. In *The 41st International Conference on Machine Learning*. 37396–37412.

[30] Jianhui Pang, Fanghua Ye, Derek Wong, Xin He, Wanshun Chen, and Longyue Wang. 2024. Anchor-based large language models. In *Findings of the Association for Computational Linguistics*. 4958–4976.

[31] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[32] Zhenmei Shi, Yifei Ming, Xuan-Phi Nguyen, Yingyu Liang, and Shafiq Joty. 2024. Discovering the gems in early layers: Accelerating long-context LLMs with 1000x input token reduction. *arXiv preprint arXiv:2409.17422* (2024).

[33] Wenqi Sun, Ruobing Xie, Junjie Zhang, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2024. Distillation is all you need for practically using different pre-trained recommendation models. *arXiv preprint arXiv:2401.00797* (2024).

[34] Qwen Team. 2024. Qwen2.5: A party of foundation models. https://qwenlm.github.io/blog/qwen2.5/

[35] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[36] Wenjie Wang, Xinyu Lin, Fuli Feng, Xiangnan He, and Tat-Seng Chua. 2023. Generative recommendation: Towards next-generation recommender paradigm. *arXiv preprint arXiv:2304.03516* (2023).

[37] Haotian Wu, Yingpeng Du, Zhu Sun, Tianjun Wei, Jie Zhang, and Ong Yew Soon. 2024. A survey on efficient solutions of large language models for recommendation. *Authorea Preprints* (2024).

[38] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.

[39] Yunjia Xi, Hangyu Wang, Bo Chen, Jianghao Lin, Menghui Zhu, Weiwen Liu, Ruiming Tang, Weinan Zhang, and Yong Yu. 2025. Efficiency unleashed: Inference acceleration for LLM-based recommender systems with speculative decoding. *arXiv preprint arXiv:2408.05676* (2025).

[40] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *The 12th International Conference on Learning Representations*.

[41] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, et al. 2024. Actions speak louder than words: trillion-parameter sequential transducers for generative recommendations. In *The 41st International Conference on Machine Learning*. 58484–58509.

[42] Yuxin Zhang, Yuxuan Du, Gen Luo, Yunshan Zhong, Zhenyu Zhang, Shiwei Liu, and Rongrong Ji. 2024. CaM: Cache merging for memory-efficient LLMs inference. In *The 41st International Conference on Machine Learning*. 58840–58850.

[43] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 1435–1448.

[44] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, et al. 2024. A survey on efficient inference for large language models. *arXiv preprint arXiv:2404.14294* (2024).

A Appendix

A.1 Detailed Analysis of Attention Score Distributions

In this section, we provide a systematic analysis of attention score distributions in LLMRec through quantitative measurements across two critical dimensions: layer order and token position. To ensure comprehensive insights, we conduct experiments across 13 NLP tasks and three real-world recommendation datasets, two mainstream LLMRec methods, and two distinct LLM architectures. Table 4 presents the overall results. More detailed quantitative results and visual figures can be found in our GitHub repository[2]. Our findings reveal fundamental differences in attention patterns between NLP tasks and LLMRec tasks, offering critical guidance for designing efficient compression strategies.

Quantitative Measurements of Attention Score Distributions. Given a sequence's attention scores $[p_1, p_2, \cdots, p_n]$, we test its sparsity by threshold-based sparsity ratio adapted from prior studies [1, 5]:

$$Sparsity = \frac{1}{n}\sum_{i=1}^{n}\mathbb{I}(p_i > \epsilon)\,, \tag{14}$$

where ϵ is the threshold. Here we set $\epsilon = 0.05$, according to the distribution characteristics of attention scores.

We test its sink by position-based total attention scores:

$$Sink_{head} = \sum_{i=1}^{T_h} p_i\,, \quad Sink_{tail} = \sum_{i=T_t}^{n} p_i\,, \tag{15}$$

where T_h and T_t is the position of the head and tail respectively. Here we set $T_h = 3, T_t = n-3$, according to the distribution characteristics of attention scores.

Attention Differences across Layer Orders. As measured in Table 4, we identify a ***layer-wise attention sparsity inversion*** between NLP tasks and LLMRec tasks. In NLP tasks, the attention sparsity is high in early layers, but decreases in later layers ($Sp_{early} > Sp_{latter}$). Conversely, LLMRec tasks display an inverted pattern: the attention sparsity is relatively low in early layers, but increases in later layers ($Sp_{early} < Sp_{latter}$). This indicates that the less sparse early layers in LLMRec retain more user preference information, while high sparsity in later layers suggests redundancy.

Attention Differences across Token Positions. Table 4 reveals a ***dual attention sinks phenomenon*** that distinguishes LLMRec from NLP tasks. There is significant $Sink_{head}$ for both tasks, while LLMRec exhibits a larger $Sink_{tail}$ ($Sink_{tail}$(LLMRec) > $Sink_{tail}$(NLP)). This indicates that both head and tail tokens in LLMRec are crucial for recommendation performance.

Implications for Compression Strategy. These findings indicate that in LLMRec, the early layers contain richer information, while the middle tokens in the later layers are redundant. Our approach takes into account the unique attention score distributions in LLMRec and formulates a reasonable compression scheme from a global perspective. It focuses on preserving information from critical token positions and early layers while pruning redundant parts in later layers. This ensures that it can improve efficiency while maintaining accuracy.

[2]https://github.com/transcend-0/EARN/blob/main/attentions.md

Table 4: Measurements of attention sink and sparsity (averaged across 13 NLP tasks, 3 real-world recommendation datasets, 2 LLMRec backbones, and 2 LLM models). Here, Sp_{early} denotes $Sparsity_{early\ layers}$, and Sp_{latter} denotes $Sparsity_{latter\ layers}$.

Model	Task	Sp_{early}	Sp_{latter}	$Sink_{head}$	$Sink_{tail}$
Llama	NLP	0.064	0.026	0.73	0.07
	LLMRec	0.025	0.048	0.56	0.09
Qwen	NLP	0.074	0.046	0.30	0.15
	LLMRec	0.047	0.059	0.30	0.24

A.2 Computational Complexity of LLM

For a decoder-only LLM with N decoder layers, the computational operations comprise three components: embedding, N stacked decoder layers, and de-embedding. Since embedding and de-embedding mainly involve lightweight lookup and projection steps, the computational bottleneck lies in the decoder layers. Specifically, each decoder layer consists of two core components: Multi-Head Attention (MHA) and Feed-Forward Network (FFN), whose FLOPs are analyzed as follows:

Multi-Head Attention (MHA). Let L denote the sequence length, d_h the hidden dimension, d_a the attention head dimension, and n_h the number of attention heads. We approximate the FLOPs of matrix multiplication $\boldsymbol{X}^{L\times d_h} \times \boldsymbol{W}^{d_h\times d_a}$ as $2Ld_hd_a$. For a single attention head, the FLOPs include: **1)** Projections for $\boldsymbol{Q}$, $\boldsymbol{K}$ and $\boldsymbol{V}$: $2Ld_hd_a \times 3 = 6Ld_hd_a$; **2)** Attention computation for $\boldsymbol{QK}^T$ and $(\cdot)\boldsymbol{V}$: $2L^2d_a + 2L^2d_a = 4L^2d_a$. Aggregating across n_h heads, plus $2Ln_hd_ad_h$ FLOPs of the final output projection, the total MHA FLOPs per decoder layer are:

$$FLOPs_{MHA} = n_h(8Ld_hd_a + 4L^2d_a)\,. \tag{16}$$

Feed-Forward Network (FFN). The FFN module involves two projections with intermediate dimension d_f: **1)** Up-projection: $2Ld_hd_f$; **2)** Down-projection: $2Ld_fd_h$. This yields total FFN FLOPs of:

$$FLOPs_{FFN} = 4Ld_hd_f\,. \tag{17}$$

Total FLOPs. Combining both components, the FLOPs for a single decoder layer are:

$$\begin{aligned} FLOPs_{LLM\ layer} &= n_h(8Ld_hd_a + 4L^2d_a) + 4Ld_hd_f \\ &= 4L[n_hd_a(2d_h + L) + d_hd_f]\,. \end{aligned} \tag{18}$$

A.3 Experimental Details

Datasets Details. For all three datasets, all historical interactions are sorted according to the global timestamps, and then split into training, validation, and testing sets with the ratio of 8:1:1. For the item identifier, we follow LC-Rec [43] and TIGER [31] to set the length $L = 4$, *i.e.,* the token sequence length of a generated item would be 4.

Implementation details. All training and inference experiments are conducted on a single NVIDIA H100 80GB GPU. For the training setup, we employ full finetuning with AdamW optimizer and an overall batch size of 128 by leveraging gradient accumulation. The learning rate is set to 0.001, and is adjusted dynamically by a cosine learning rate scheduler with a warmup ratio of 0.02. For the inference setup, we set the beam size to 20 and use the maximum batch size that the GPU could accommodate. For the

Table 5: Overall performance comparison between the baselines and EARN instantiated on TIGER.

Method	Time Efficiency ω	Time Efficiency τ	Space Efficiency γ	Space Efficiency σ	Recommendation Effectiveness R@10	R@20	N@10	N@20
Beauty, Llama								
Finetune	1.00	602.50	0.0	68.18	0.0108	0.0198	0.0071	0.0096
SkipLayers	1.78	1069.7	44.5	37.87	0.0010	0.0010	0.0011	0.0011
POD	1.17	701.7	14.8	58.11	0.0033	0.0065	0.0027	0.0039
500xCompressor	2.31	1389.1	74.8	17.16	0.0003	0.0003	0.0001	0.0001
StreamingLLM	1.21	730.0	**96.4**	**2.44**	0.0003	0.0005	0.0005	0.0003
SnapKV	1.19	718.1	94.5	3.76	0.0041	0.0053	0.0027	0.0031
EARN	**3.44**	**1972.4**	80.9	13.04	**0.0115**	**0.0199**	**0.0075**	**0.0098**
Beauty, Qwen								
Finetune	1.00	714.3	0.0	12.72	0.0095	0.0162	0.0061	0.0081
SkipLayers	1.69	1208.1	57.9	5.35	0.0000	0.0000	0.0000	0.0000
POD	1.10	784.4	8.4	11.65	0.0056	0.0084	0.0032	0.0044
500xCompressor	2.55	1819.2	91.4	1.09	0.0003	0.0005	0.0002	0.0005
StreamingLLM	1.05	751.6	**91.7**	**1.06**	0.0057	0.0096	0.0040	0.0052
SnapKV	1.02	728.7	69.6	3.87	0.0063	0.0110	0.0043	0.0055
EARN	**2.58**	**1805.9**	75.5	3.11	**0.0104**	**0.0164**	**0.0065**	**0.0089**
Games, Llama								
Finetune	1.00	695.8	0.0	61.40	0.0126	**0.0216**	0.0080	0.0107
SkipLayers	1.83	1272.4	45.0	33.76	0.0001	0.0001	0.0003	0.0003
POD	1.08	757.8	15.8	51.67	0.0054	0.0098	0.0032	0.0045
500xCompressor	2.11	1470.2	72.5	16.88	0.0010	0.0015	0.0007	0.0008
StreamingLLM	1.22	850.7	**96.0**	**2.46**	0.0010	0.0010	0.0010	0.0009
SnapKV	1.21	837.6	93.5	3.98	0.0064	0.0075	0.0045	0.0045
EARN	**3.12**	**2062.7**	81.2	11.56	**0.0138**	0.0212	**0.0085**	**0.0108**
Games, Qwen								
Finetune	1.00	673.2	0.0	10.81	0.0131	0.0224	0.0085	0.0113
SkipLayers	1.51	1014.3	57.0	4.65	0.0000	0.0000	0.0000	0.0000
POD	1.36	912.1	8.5	9.88	0.0087	0.0150	0.0050	0.0073
500xCompressor	2.60	1752.0	96.7	0.35	0.0002	0.0004	0.0001	0.0001
StreamingLLM	1.06	708.8	**98.4**	**0.17**	0.0088	0.0145	0.0050	0.0072
SnapKV	1.02	686.4	67.6	3.50	0.0094	0.0161	0.0056	0.0081
EARN	**2.86**	**1841.3**	72.1	3.02	**0.0137**	**0.0231**	**0.0086**	**0.0119**
MovieLens, Llama								
Finetune	1.00	852.50	0.0	39.79	0.0245	0.0446	0.0244	0.0323
SkipLayers	1.85	1577.0	67.6	12.89	0.0022	0.0024	0.0059	0.0051
POD	1.18	1003.9	18.1	32.57	0.0068	0.0122	0.0088	0.0103
500xCompressor	1.93	1640.0	61.1	15.48	0.0004	0.0006	0.0004	0.0003
StreamingLLM	1.18	1007.9	**95.2**	**1.89**	0.0000	0.0000	0.0000	0.0000
SnapKV	1.18	1002.50	91.8	3.28	0.0000	0.0000	0.0000	0.0000
EARN	**2.85**	**2437.4**	77.7	8.87	**0.0269**	**0.0544**	**0.0221**	**0.0340**
MovieLens, Qwen								
Finetune	1.00	897.4	0.0	7.75	0.0238	0.0455	0.0224	0.0312
SkipLayers	1.49	1340.5	76.9	1.79	0.0002	0.0002	0.0002	0.0002
POD	1.41	1260.9	39.8	4.66	0.0114	0.0192	0.0106	0.0131
500xCompressor	2.10	1882.1	77.9	1.71	0.0004	0.0009	0.0005	0.0007
StreamingLLM	1.01	912.2	**88.3**	**0.91**	0.0213	0.0402	0.0180	0.0241
SnapKV	0.97	872.4	76.5	1.82	0.0204	0.0464	0.0174	0.0263
EARN	**2.56**	**2292.8**	45.5	4.23	**0.0274**	**0.0474**	**0.0273**	**0.0355**

Table 6: Overall comparison between baselines and EARN instantiated on HSTU. Here, the unit of σ is MB.

Method	Time Efficiency ω	Time Efficiency τ	Space Efficiency γ	Space Efficiency σ	Recommendation Effectiveness R@10	R@20	N@10	N@20
Beauty								
Finetune	1.00	2735.5	0.0	439.5	0.0446	**0.0693**	0.0269	0.0328
SkipLayers	1.72	4685.6	27.8	317.5	0.0248	0.0322	0.0116	0.0136
EARN	**2.01**	**5414.9**	**49.5**	**222.0**	**0.0520**	0.0594	**0.0311**	**0.0329**
Games								
Finetune	1.00	3425.2	0.0	532.9	0.0557	**0.0846**	0.0362	**0.0435**
SkipLayers	1.69	5797.2	27.9	384.4	0.0371	0.0520	0.0200	0.0237
EARN	**1.97**	**6770.9**	**50.3**	**264.9**	**0.0586**	0.0735	**0.0384**	0.0421
MovieLens								
Finetune	1.00	3187.9	0.0	546.9	0.0756	0.1073	0.0405	0.0485
SkipLayers	1.75	5602.4	34.6	357.7	0.0780	0.1207	0.0429	0.0536
EARN	**2.06**	**6555.8**	**50.5**	**270.6**	**0.0793**	**0.1268**	**0.0429**	**0.0551**

Table 7: Overall performance comparison between Finetune and EARN with different register layer k on Llama (total 32 layers) on MMLU.

RegisterLayer k	ω	τ	γ	σ	Accuracy
Finetune	1.0	76.0	0.0	266.0	**0.51**
4	**6.9**	**523.6**	**83.3**	**44.5**	0.27
7	4.3	331.3	73.3	71.0	0.28
11	2.9	219.0	61.3	103.0	0.28
15	2.2	163.4	49.3	135.0	0.46
19	1.7	130.3	38.2	164.5	0.50
23	1.4	104.3	26.1	196.5	0.50
27	1.2	91.6	14.2	228.3	0.50

hyper-parameters of EARN in Table 1, EARN on Llama uses $k = 4$ and $n = 1$, and EARN on Qwen uses $k = 7$ and $n = 1$.

A.4 Additional Results on TIGER

Table 5 shows the overall performance comparison between the baselines and EARN instantiated on TIGER. Our EARN achieves the best performance in terms of time efficiency and recommendation effectiveness, while also demonstrating excellent space efficiency. These results further validate the effectiveness of EARN.

A.5 Additional Results on HSTU

We also conduct experiments on HSTU [41], the first industry-deployed generative recommendation system. As shown in Table 6, our EARN is also effective for HSTU, demonstrating excellent time and space efficiency (2x speedup and 50% memory savings), with minimal impact on recommendation effectiveness. It is important to note that HSTU differs from other popular LLM-based generative recommendation methods (*e.g.*, LC-Rec). Specifically, in HSTU model, there are no textual inputs and no concept of prompts as seen in NLP tasks. And HSTU directly generates the predicted item embeddings, akin to generating only one token, which means it only has the prefilling stage and no decoding stage. These render that the baselines of prompt compression, cache compression, and gist methods in NLP are not applicable. Therefore, we primarily compared Finetune, SkipLayer, and EARN.

A.6 Additional Results on NLP Tasks

To validate the generalizability of EARN beyond recommendation tasks, we conducted experiments on MMLU dataset [8]—a general NLP dataset spanning 57 tasks (e.g., math, science, and humanities).

Results in Table 7 demonstrate EARN's effectiveness: **1)** Efficiency-accuracy trade-off: As the register layer k decreases, EARN achieves higher speedup and cache reduction at the cost of reduced accuracy. However, when $k \geq 15$, the accuracy loss remains within 10%, indicating that EARN can still deliver satisfactory performance for general NLP tasks. **2)** Comparison with LLMRec tasks: Unlike LLM-Rec tasks, where EARN incurs no accuracy loss at $k \geq 4$ and even boosts accuracy by pruning noisy information (refer to Figure 6), NLP tasks require a higher k to curb accuracy degradation.

In summary, while optimal k varies across tasks, it consistently enables efficient inference with controlled accuracy trade-offs, showcasing its broader applicability beyond recommendation tasks.